AUBURN

UNIVERSITY

File System

Overview

Objectives

- Review the mass storage (disk) structure and issues.
- Learn and understand the disk scheduling.
- Learn and understand the disk allocation methods

Requirements

- Basics from Computer Organization principles (COMP 3350 or equivalent)
- Familiar with file interface.
- Complete reading assignments:
  - Overview Mass Storage
  - Disk structure
  - Disk scheduling
  - (Disk) allocation methods

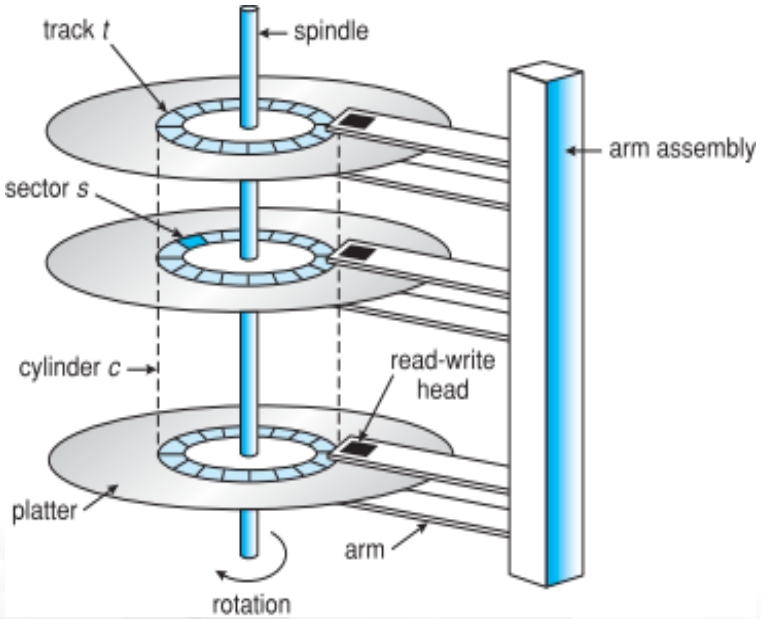
Disk Structure

- **Objectives**
  1. Describe the physical structure of secondary storage
  2. Understand the effects of the structure
  3. Present the performance characteristics

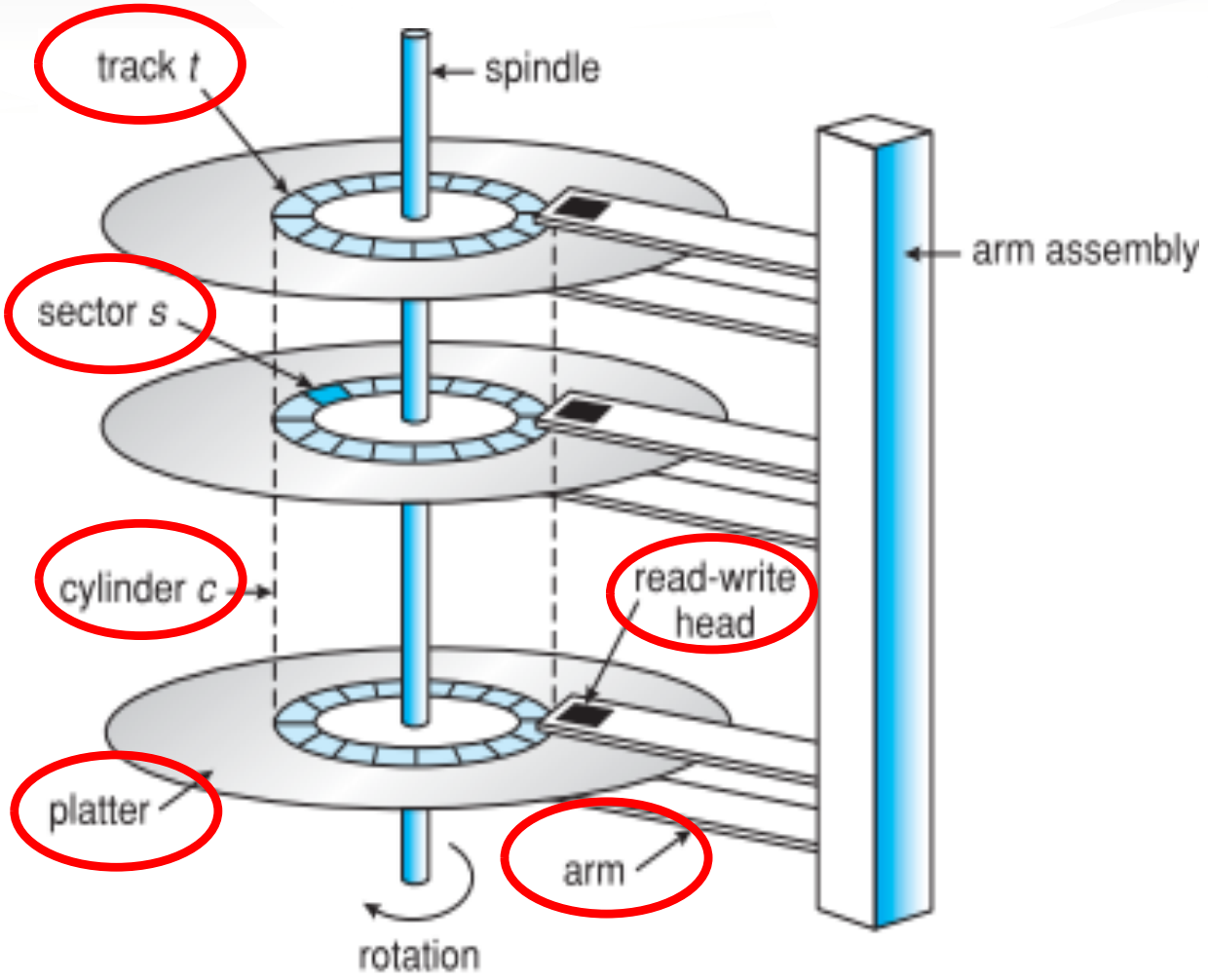
- Read (From Textbook))
  - Overview of Mass-Storage Structure
  - Disk Structure

Overview of Mass Storage Structure

- **Magnetic disks** provide bulk of secondary storage of modern computers
  - Drives rotate at 60 to 250 times per second
  - **Transfer rate** is rate at which data flow between drive and computer
  - **Positioning time** (**random-access time**) is time to move disk arm to desired cylinder (**seek time**) and time for desired sector to rotate under the disk head (**rotational latency**)
  - **Head crash** results from disk head making contact with the disk surface -- That's bad
- Disks can be removable
- Drive attached to computer via **I/O bus**
  - Busses vary, including **EIDE**, **ATA**, **SATA**, **USB**, **Fibre Channel**, **SCSI, SAS, Firewire**
  - **Host controller** in computer uses bus to talk to **disk controller** built into drive or storage array



Moving-head Disk Mechanism



Hard Disks

- Platters range from .85” to 14” (historically)
  - Commonly 3.5”, 2.5”, and 1.8”
- Range from 30GB to 3TB per drive
- Performance
  - Transfer Rate – theoretical – 6 Gb/sec
  - Effective Transfer Rate – real – 1Gb/sec
  - Seek time from 3ms to 12ms – 9ms common for desktop drives
  - (**Worst**) Latency based on spindle (rotational) speed
    - Time to perform one rotation
    - Convert speed from per minute to per second: w /60
    - Find the time to perform one rotation: 1/(w/60)
  - **Average** latency = ½ worst latency
    - Example: what is the average latency if spindle rotational speed is 15000 rpm
    - Step 1: convert speed to rotation per second: rps = 15000/60 = 250 rotations per second
    - Step 2: worst latency = time to complete one rotation = 1 second/250 = 4 ms
    - Step 3: average latency = ½ worst latency = ½ 4ms = 2ms

Spindle [rpm]	Average latency [ms]
4200	7.14
5400	5.56
7200	4.17
10000	3
15000	2

(From Wikipedia)

Hard Disk Performance

- **Average access time** = average seek time + average latency
  - For fastest disk 3ms + 2ms = 5ms
  - For slow disk 9ms + 5.56ms = 14.56ms
- Recall:
  - seek time is related to head moves
  - latency related to platter/disk rotation
  - Access time is only the time to position the head at the beginning of a sector
  - We must add transfer time and disk controller overhead
- Average I/O time = average access time + transfer time + controller overhead
- transfer time = (amount to transfer / transfer rate)
- For example to transfer a 4KB block on a 7200 RPM disk with a 5ms average seek time, 1Gb/sec transfer rate with a .1ms controller overhead =
  - 5ms + 4.17ms + transfer time + 0.1ms =
  - Transfer time = 4KB / 1Gb/s * 8Gb = 0.032 ms
  - Average I/O time for 4KB block = 9.27ms + .032ms = 9.302ms

Solid-State Disks

- Nonvolatile memory used like a hard drive
  - Many technology variations
- Can be more reliable than HDDs
- More expensive per MB
- Maybe have shorter life span
- Less capacity
- But much faster
- Busses can be too slow -> connect directly to PCI for example
- No moving parts, so no seek time or rotational latency

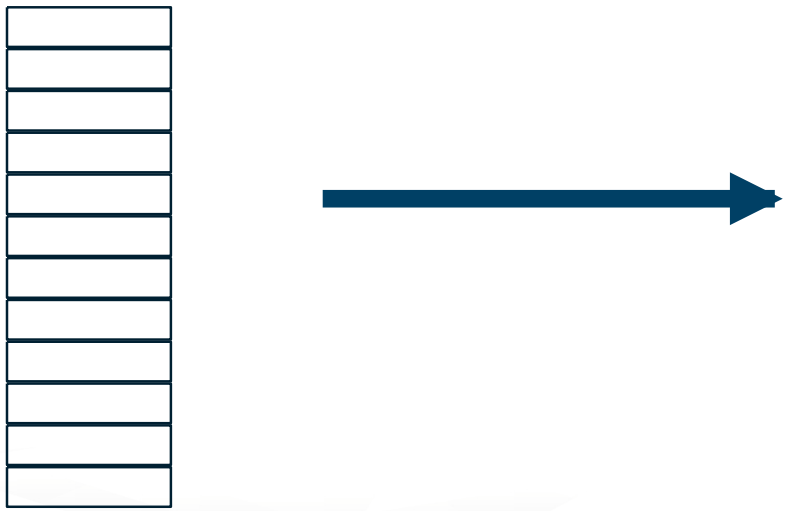
Magnetic Tape

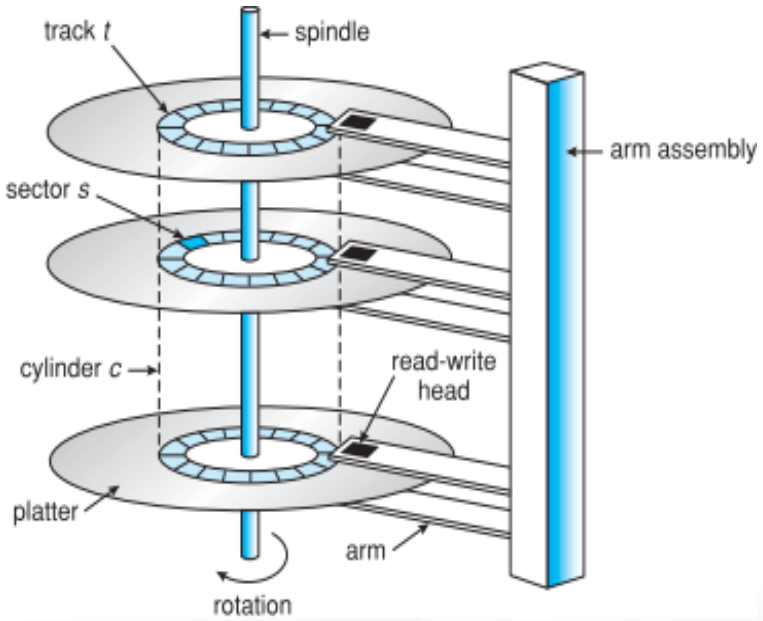
- Was early secondary-storage medium
  - Evolved from open spools to cartridges
- Relatively permanent and holds large quantities of data
- Access time **slow**
- Random access ~1000 times slower than disk
- Mainly used for backup, storage of infrequently-used data, transfer medium between systems
- Kept in spool and wound or rewound past read-write head
- Once data under head, transfer rates comparable to disk
  - 140MB/sec and greater
- 200GB to 1.5TB typical storage
- Common technologies are LTO-{3,4,5} and T10000

Disk Structure

- Disk drives are addressed as large 1-dimensional arrays of **logical blocks**, where the logical block is the smallest unit of transfer
  - Low-level formatting creates **logical blocks** on physical media
- The 1-dimensional array of logical blocks is mapped into the sectors of the disk sequentially
  - Sector 0 is the first sector of the first track on the outermost cylinder
  - Mapping proceeds in order through that track, then the rest of the tracks in that cylinder, and then through the rest of the cylinders from outermost to innermost
  - Logical to physical address should be easy
    - Except for bad sectors
    - Non-constant # of sectors per track via constant angular velocity

l-dimensional array



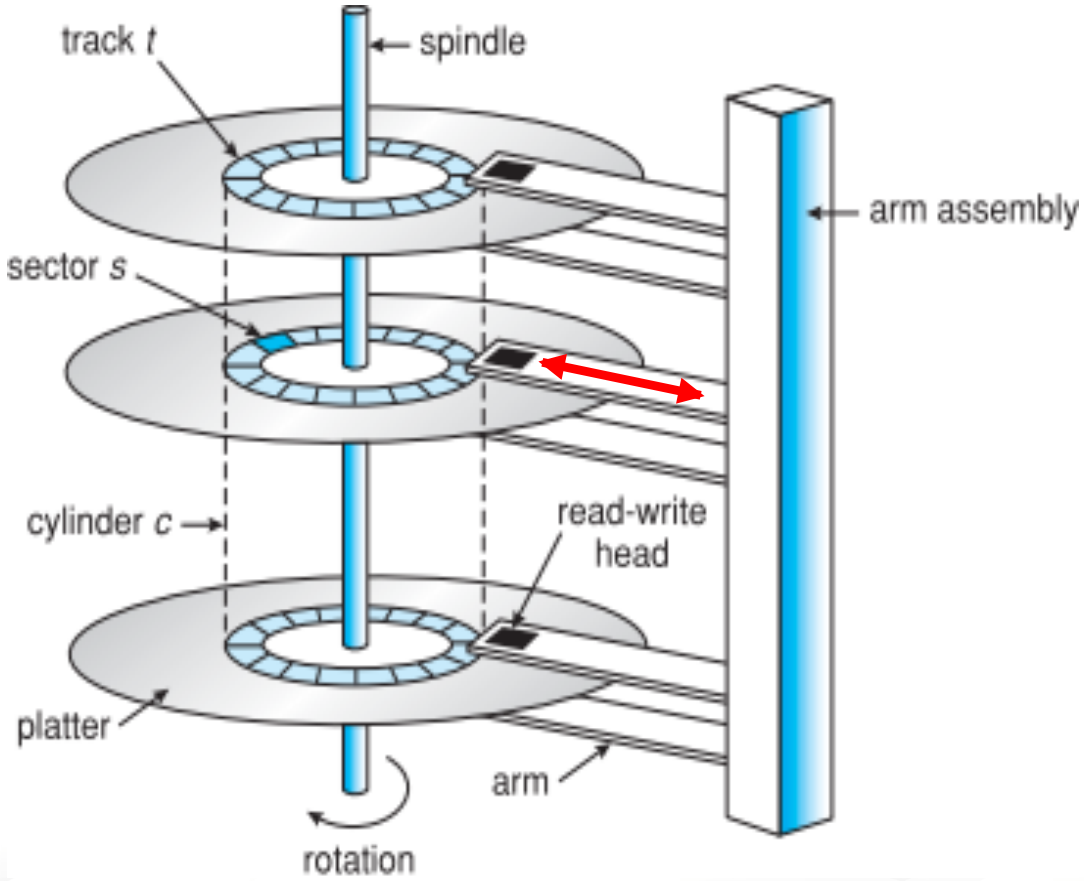


Disk Scheduling (1/3)

- Objective
  - Minimize seek time
- How to minimize seek time?
  - Sort block requests to minimize head moves
  - “Sorting” is scheduling
- Disk Scheduling Policies
  - First Come First Serve
  - Shortest Seek Time First
  - SCAN
  - C-SCAN
  - LOOK
  - C-LOOK
- Read “Disk Scheduling” Section

Disk Scheduling

- The operating system is responsible for using hardware efficiently — for the disk drives, this means having a fast access time and disk bandwidth
- Minimize seek time
- Seek time ? seek distance



Disk Scheduling (Cont.)

- There are many sources of disk I/O request
  - OS
  - System processes
  - Users processes
- I/O request includes input or output mode, disk address, memory address, number of sectors to transfer
- OS maintains queue of requests, per disk or device
- Idle disk can immediately work on I/O request, busy disk means work must queue
  - Optimization algorithms only make sense when a queue exists

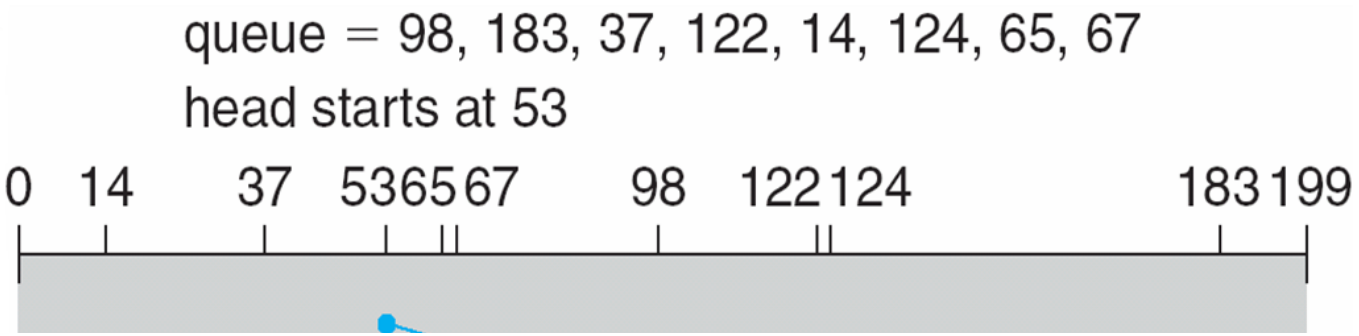
Disk Scheduling (Cont.)

- Note that drive controllers have small buffers and can manage a queue of I/O requests (of varying “depth”)
- Several algorithms exist to schedule the servicing of disk I/O requests
- The analysis is true for one or many platters
- We illustrate scheduling algorithms with a request queue (0-199)

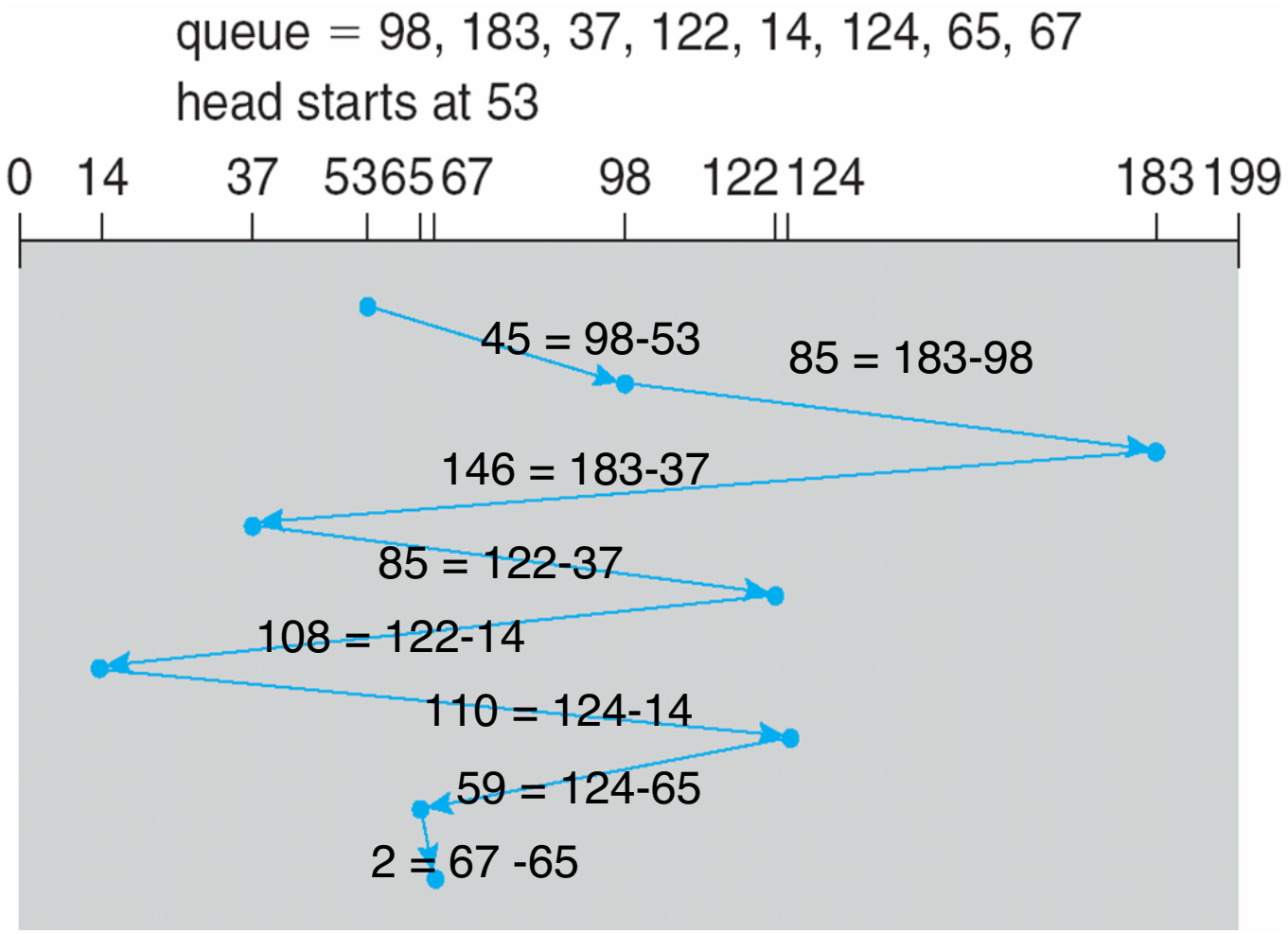
98, 183, 37, 122, 14, 124, 65, 67

Head pointer is at Position 53

FCFS


queue = 98, 183, 37, 122, 14, 124, 65, 67
head starts at 53
0
14
37
53
65
67
98
122
124
183
199

FCFS



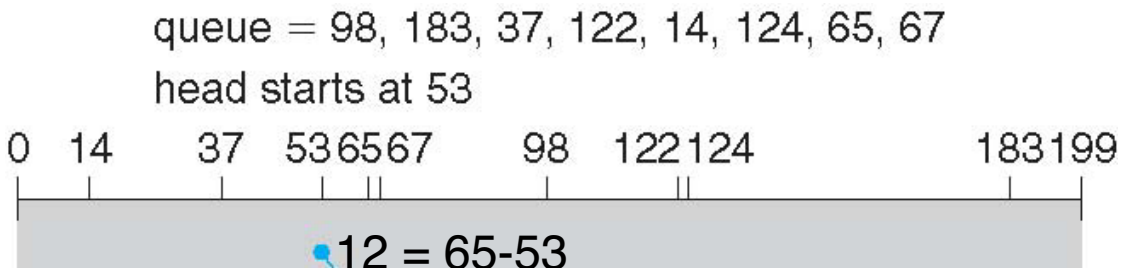
45 + 85 + 146 + 85 + 108 + 110 + 59 + 2 = 640

Disk Scheduling (2/3)

- Objective
  - Minimize seek time
- How to minimize seek time?
  - Sort requests to minimize head moves
  - “Sorting” is scheduling
- Disk Scheduling Policies
  - First Come First Serve
  - Shortest Seek Time First
  - SCAN
  - C-SCAN
  - LOOK
  - C-LOOK
- Read “Disk Scheduling” Section

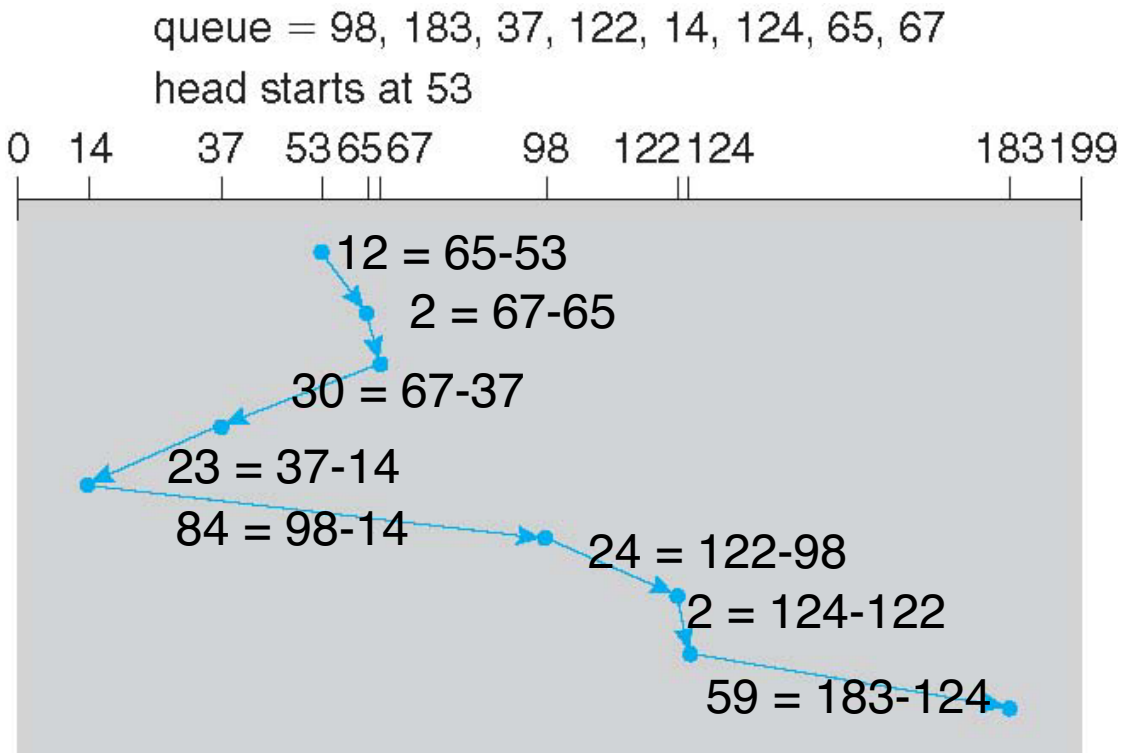
SSTF

- Shortest Seek Time First selects the request with the minimum seek time from the current head position
- SSTF scheduling is a form of SJF scheduling; may cause **starvation** of some requests
- **Illustration shows total head movement of 236 cylinders**



SSTF

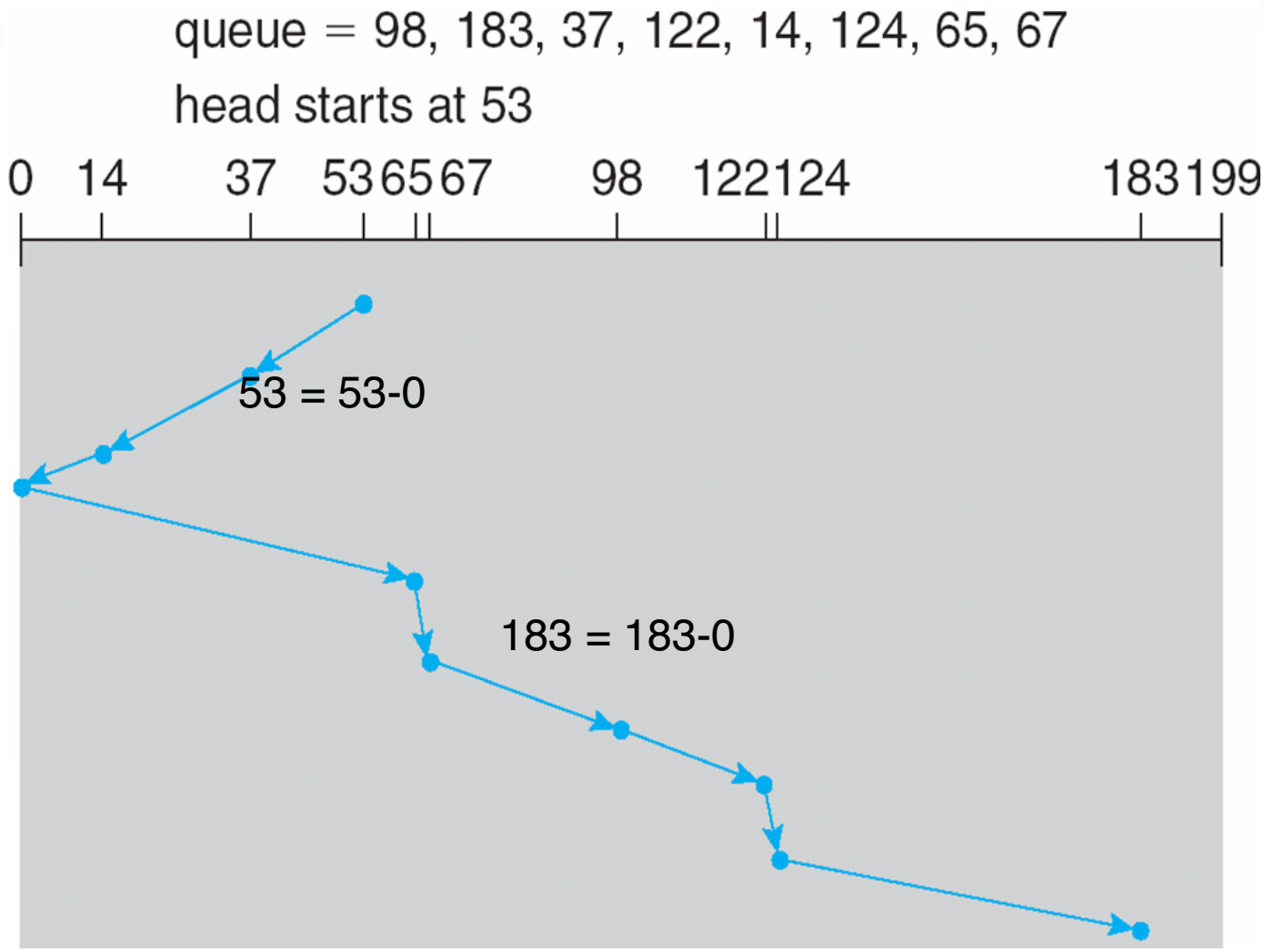
- Shortest Seek Time First selects the request with the minimum seek time from the current head position
- SSTF scheduling is a form of SJF scheduling; may cause **starvation** of some requests
- **Illustration shows total head movement of 236 cylinders**



SCAN

- The disk arm starts at one end of the disk, and moves toward the other end, servicing requests until it gets to the other end of the disk, where the head movement is reversed and servicing continues.
- **SCAN algorithm** Sometimes called the **elevator algorithm**
- Illustration shows total head movement of 208 cylinders
- But note that if requests are uniformly dense, largest density at other end of disk and those wait the longest

SCAN (Cont.)



Total Head Moves = 53 + 183 = 236

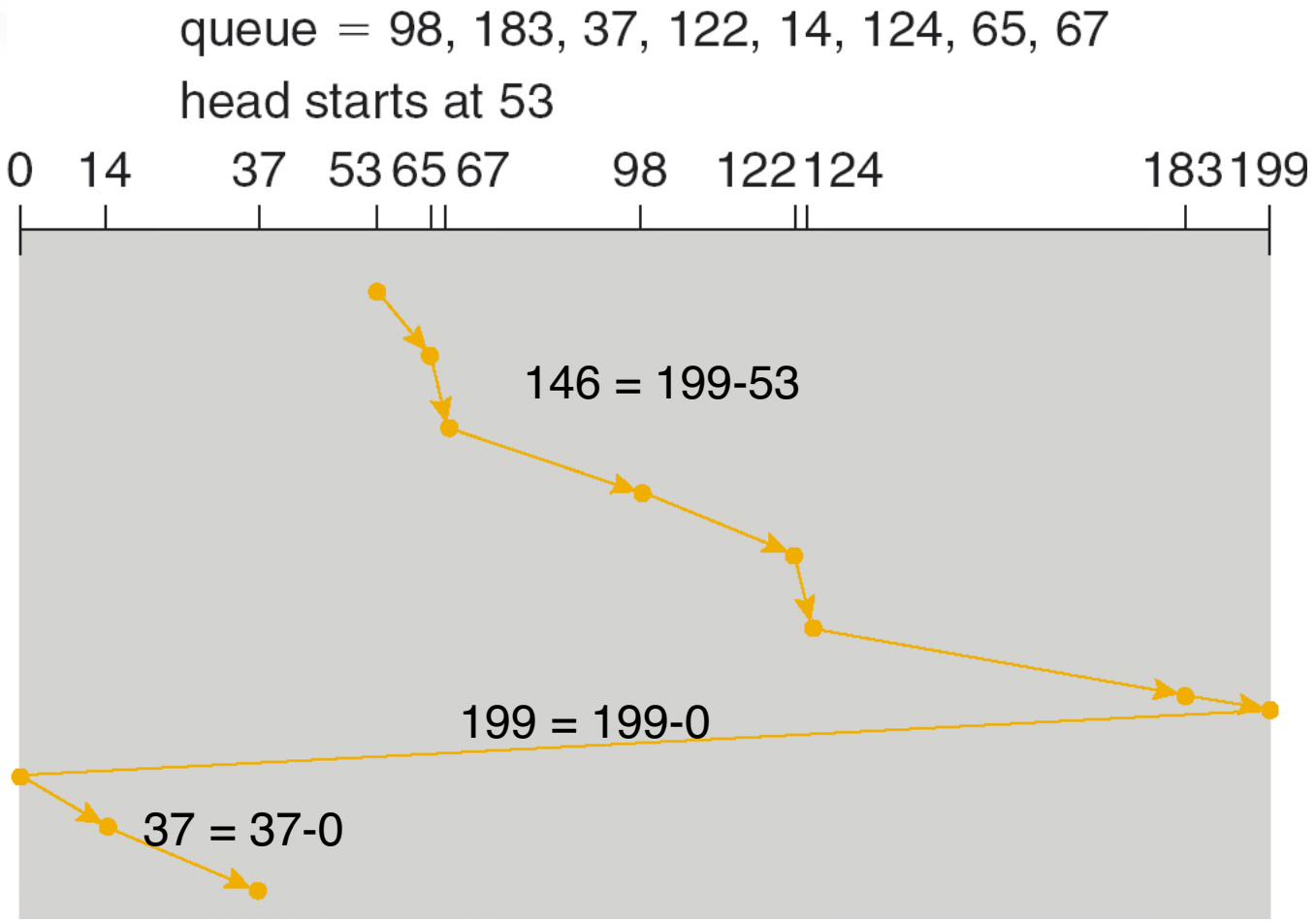
Disk Scheduling (3/3)

- Objective
  - Minimize seek time
- How to minimize seek time?
  - Sort requests to minimize head moves
  - “Sorting” is scheduling
- Disk Scheduling Policies
  - First Come First Serve
  - Shortest Seek Time First
  - SCAN
  - C-SCAN
  - LOOK
  - C-LOOK
- Read “Disk Scheduling” Section

C-SCAN

- Provides a more uniform wait time than SCAN
- The head moves from one end of the disk to the other, servicing requests as it goes
  - When it reaches the other end, however, it immediately returns to the beginning of the disk, without servicing any requests on the return trip
- Treats the cylinders as a circular list that wraps around from the last cylinder to the first one

C-SCAN (Cont.)



Total Head Moves = 146 + 199 + 37 = 382

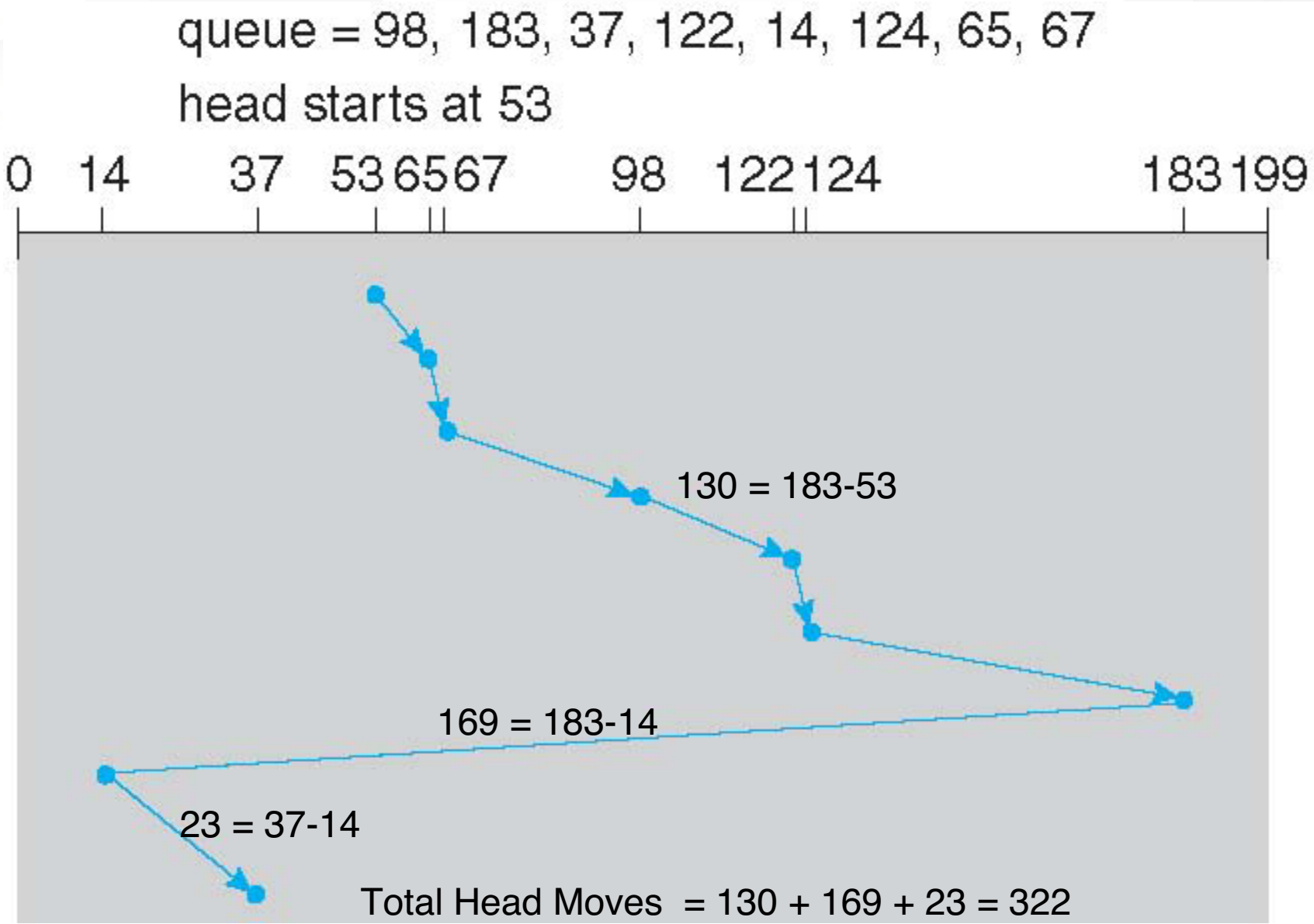
LOOK

- Improvement of SCAN
- Problem with SCAN?
  - Unnecessary moves of the head to extremeties
- LOOK: Arm only goes as far as the last request in each direction, then reverses direction immediately, without first going all the way to the end of the disk
- Total number of cylinders?

C-LOOK

- Arm only goes as far as the last request in each direction, then reverses direction immediately, without first going all the way to the end of the disk
- …. and serves requests only in one direction (like C-SCAN)

C-LOOK (Cont.)


queue = 98, 183, 37, 122, 14, 124, 65, 67
head starts at 53
0
14
37
53
65
67
98
122
124
183
199
130 = 183-53
169 = 183-14
23 = 37-14
Total Head Moves = 130 + 169 + 23 = 322

Selecting a Disk-Scheduling Algorithm

- SSTF is common and has a natural appeal
- SCAN and C-SCAN perform better for systems that place a heavy load on the disk
  - Less starvation
- Performance depends on the number and types of requests
- Requests for disk service can be influenced by the file-allocation method
  - And metadata layout
- The disk-scheduling algorithm should be written as a separate module of the operating system, allowing it to be replaced with a different algorithm if necessary
- Either SSTF or LOOK is a reasonable choice for the default algorithm
- What about rotational latency?
  - Difficult for OS to calculate
- How does disk-based queueing effect OS queue ordering efforts?

Allocation Methods

- Objective
  - How to allocate blocks for a file?
- Allocation Methods
  - Contiguous allocation
  - Linked allocation
    - File Allocation Table (FAT)
  - Indexed allocation
  - Combined (indexed) scheme: Unix file system

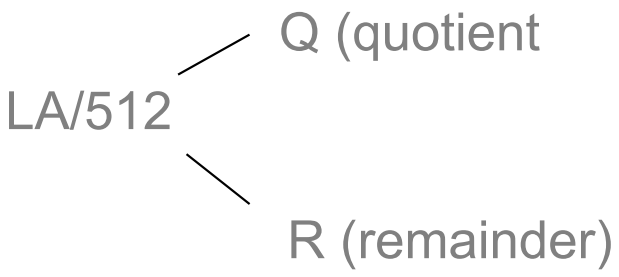
- Read “Allocation Methods” Section

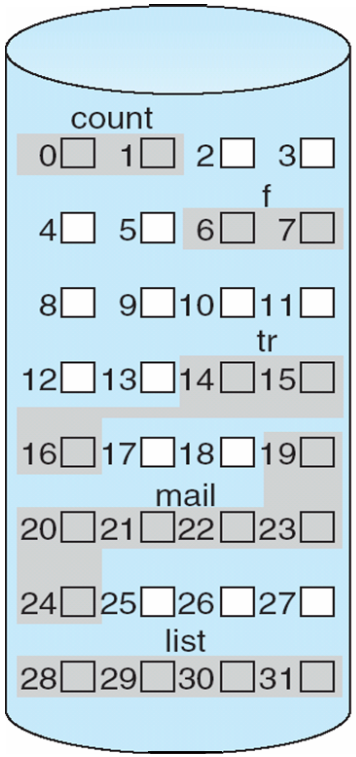
Allocation Methods - Contiguous

- An allocation method refers to how disk blocks are allocated for files:
- **Contiguous allocation** – each file occupies set of contiguous blocks
  - Best performance in most cases
  - Simple – only starting location (block #) and length (number of blocks) are required
  - Problems include finding space for file, knowing file size, **external** fragmentation, need for **compaction off-line** (**downtime**) or **on-line**
  - Is this allocation method good for sequential and random access?

Contiguous Allocation

- Mapping from logical to physical (LA is Logical Address)





directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

Block to be accessed = Q + starting address
Displacement into block = R

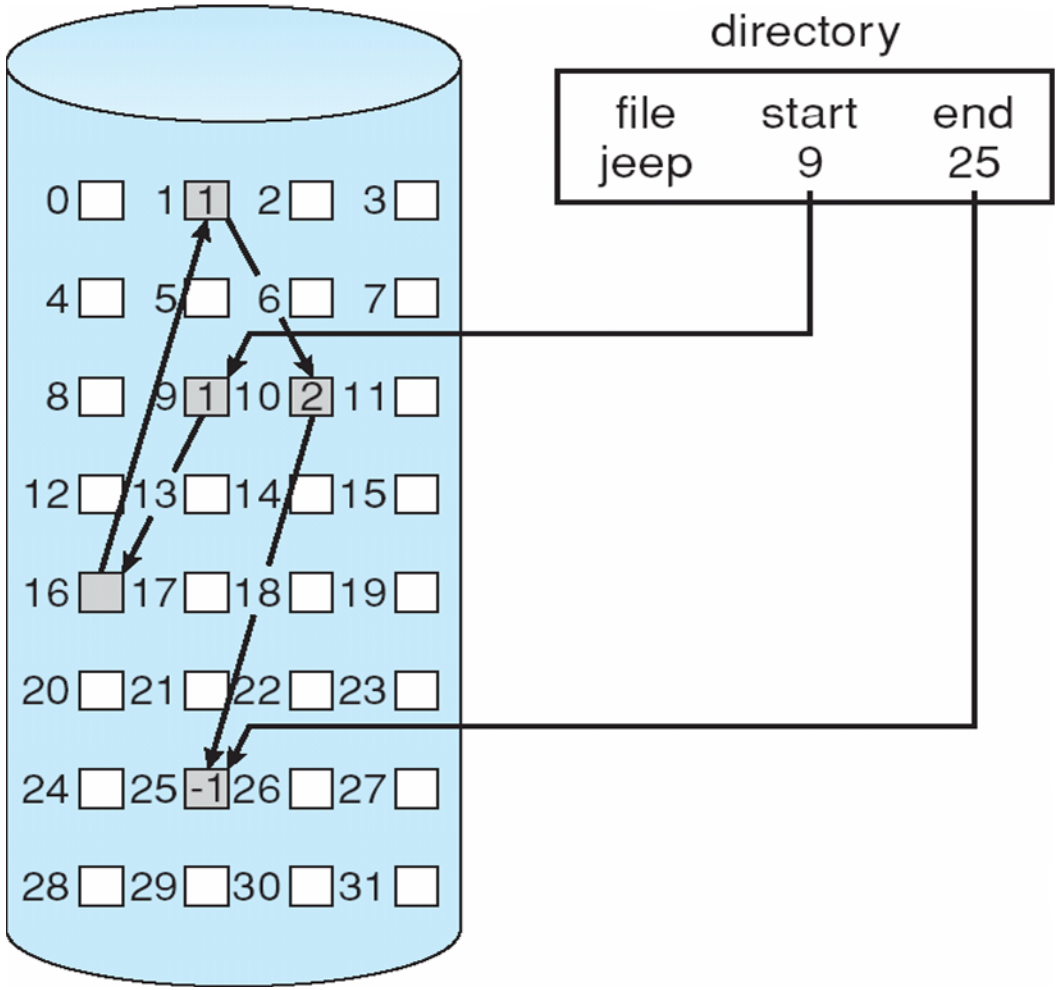
Extent-Based Systems

- Many newer file systems (i.e., Veritas File System) use a modified contiguous allocation scheme
- Extent-based file systems allocate disk blocks in extents
- An **extent** is a contiguous block of disks
  - Extents are allocated for file allocation
  - A file consists of one or more extents

Allocation Methods - Linked

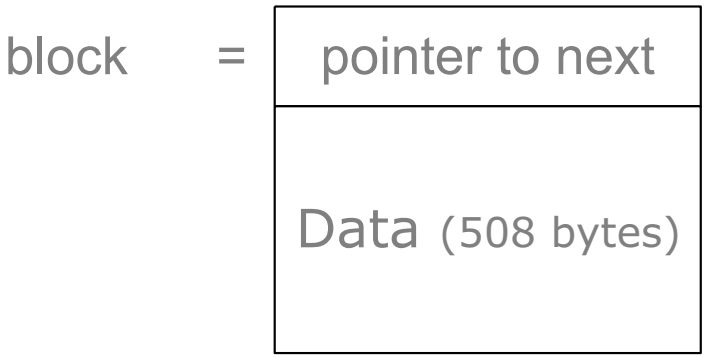
- **Linked allocation** – each file a linked list of blocks
  - File ends at nil pointer
  - No external fragmentation
  - Each block contains pointer to next block
  - No compaction, external fragmentation
  - Improve efficiency by clustering blocks into groups but increases internal fragmentation
  - Reliability can be a problem
  - Locating a block can take many I/Os and disk seeks
  - Is this allocation method good for sequential and random access?

Linked Allocation

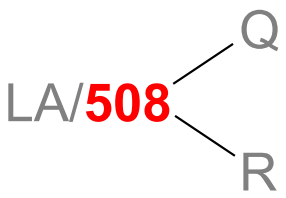


Linked Allocation

- Each file is a linked list of disk blocks: blocks may be scattered anywhere on the disk



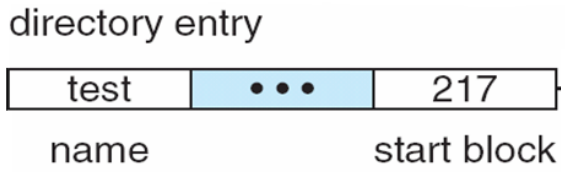
- Mapping from logical to physical (LA is Logical Address)



  - Block to be accessed is the Qth block in the linked chain of blocks representing the file.
  - Key issue: each block must be read until block found
  - Displacement into block = R + 4 (jump the pointer)

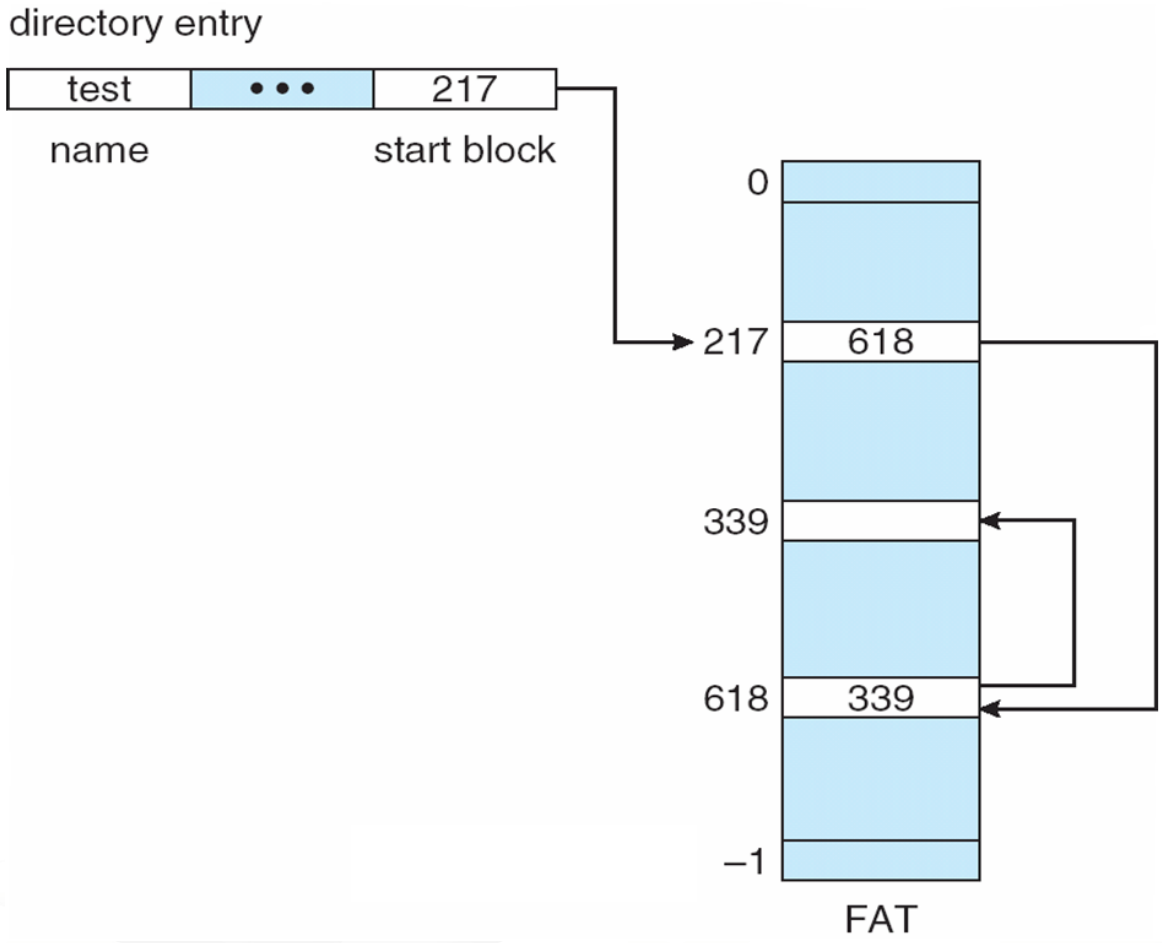
File Allocation Table (FAT)– (Linked Allocation Variation)

- FAT (File Allocation Table) variation
  - Beginning of volume has table, indexed by block number
  - Much like a linked list, but faster on disk and cacheable
  - New block allocation simple



File Allocation Table (FAT)– (Linked Allocation Variation)

- FAT (File Allocation Table) variation
  - Beginning of volume has table, indexed by block number
  - Much like a linked list, but faster on disk and cacheable
  - New block allocation simple



Allocation Methods

- Objective
  - How to allocate blocks for a file?
- Allocation Methods
  - Contiguous allocation
  - Linked allocation
    - File Allocation Table (FAT)
  - Indexed allocation
  - Combined (indexed) scheme: Unix file system

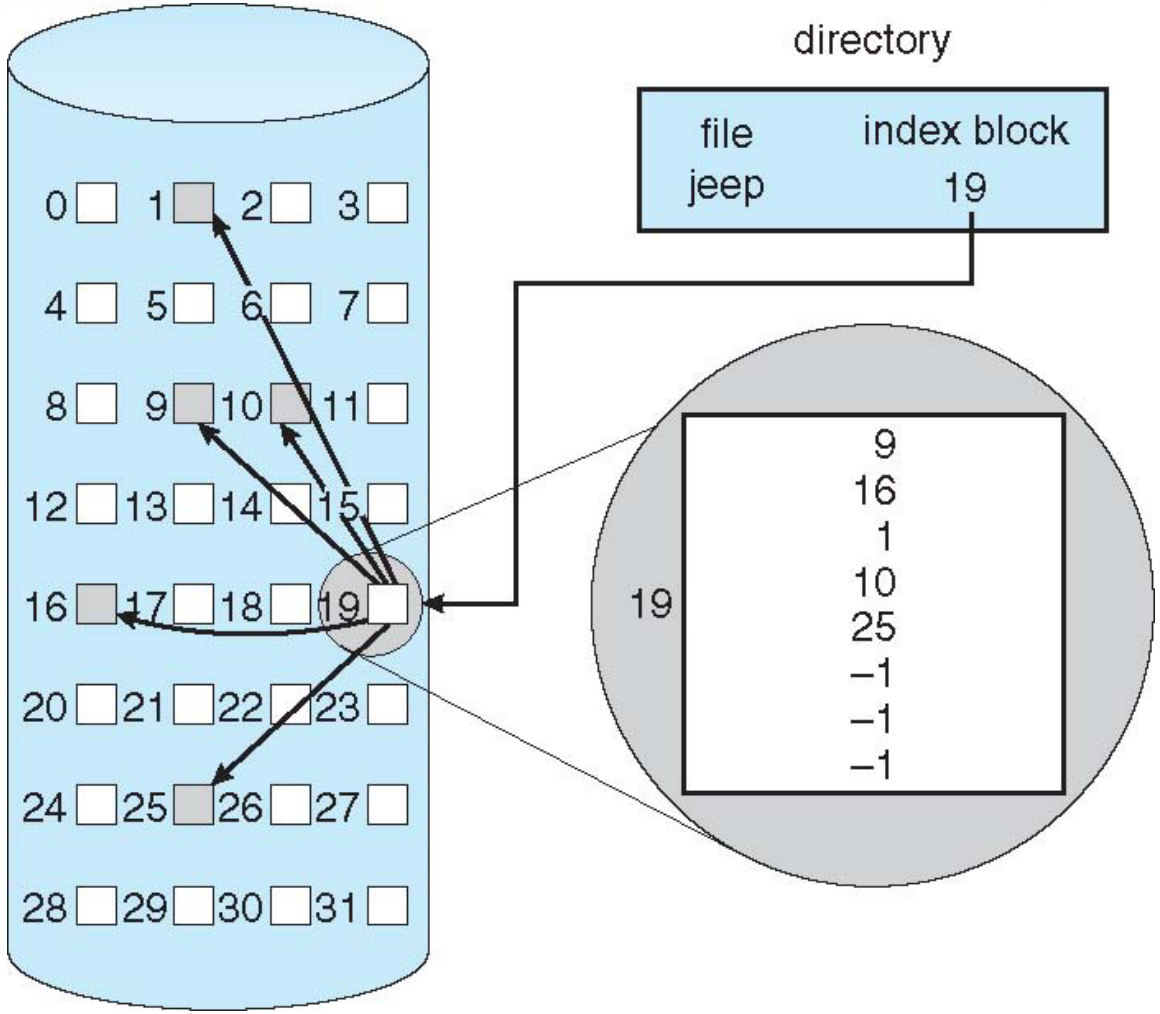
- Read Section “Allocation Methods” Section

Allocation Methods - Indexed

- **Indexed allocation**
  - Each file has its own **index block**(s) of pointers to its data blocks

- Logical view

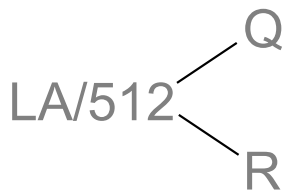
Example of Indexed Allocation



Indexed Allocation (Cont.)

- Need index table
- Random access
- Dynamic access without external fragmentation, but have overhead of index block
- Is this allocation method good for sequential and random access?

- **Mapping from logical to physical**
  - **Example:** block size = 512 bytes, File maximum size : 64 KB. Index table takes one block.
  - A 64KB contains 64KB/512 blocks = $2^{16}/2^{9}$ blocks = 2^{7} blocks = 128 blocks
  - Each block pointer takes 4 bytes
  - You need 4*128 bytes to store 128 pointers = 512 bytes = 1 block



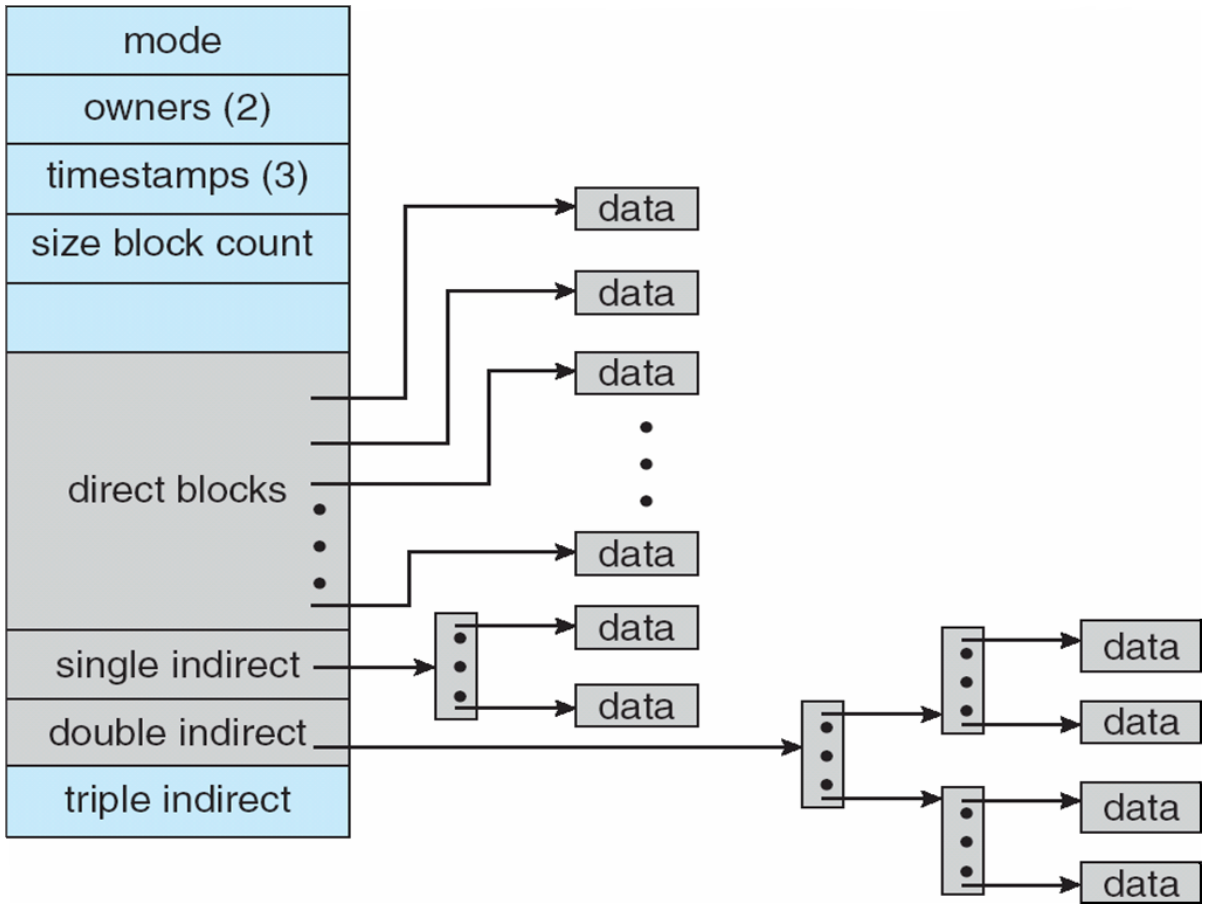
- Q = displacement into index table
- R = displacement into block

Indexed Allocation

- Simple indexed allocation
- Multilevel indexed allocation
- Combined scheme (direct and multilevel): Unix file system

Combined Scheme: UNIX UFS

4K bytes per block, 32-bit addresses



More index blocks than can be addressed with 32-bit file pointer

Performance

- Best method depends on file access type (random or sequential)
  - Contiguous great for sequential and random
- Linked good for sequential, not random
- Declare access type at creation -> select either contiguous or linked
- Indexed more complex
  - Single block access could require 2 index block reads then data block read
  - Clustering can help improve throughput, reduce CPU overhead

Wrap Up

- Review the mass storage (disk) structure and issues.
  - Platters, arm, head…
  - Relationship structure and access time
  - seek time, average latency
- Learn and understand the disk scheduling to hierarchical memory.
  - FCFS
  - SSTF
  - SCAN, C-SCAN
  - LOOK, C-LOOK
- Describe, discuss and evaluate the disk space allocations methods.
  - Contiguous
  - Linked
  - Indexed
    - Simple
    - Multilevel
    - Combined